

# 2024 年非专业级别软件能力认证第一轮

## (CSP-J) 入门级 C++语言模拟试题

### 考生注意事项:

试题纸共有 页, 答题纸共有 1 页, 满分 100 分。请在答题纸上作答, 写在试题纸上的一律无效。

不得使用任何电子设备(如计算器、手机、电子词典等)或查阅任何书籍资料。

### 一、单项选择题(共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1. 以下断电之后仍能保存数据的有( B )。

- A. 寄存器      B. ROM      C. RAM      D. 高速缓存

2. 编写一个程序一般需要经过的几个步骤依次是( B )。

- A. 编译、编辑、链接、调试、运行  
B. 编辑、编译、链接、运行、调试  
C. 编译、运行、调试、编辑、链接  
D. 编辑、调试、编辑、链接、运行

3. 十进制数 100.625 等值于二进制数( D )

- A. 1001100.101      B. 1100100.101  
C. 1100100.011      D. 1001100.11

4. 十进制算术表达式:  $3 \times 512 + 7 \times 64 + 4 \times 8 + 5$  的运算结果, 用二进制表示为( B )。

- A. 10111100101      B. 11111100101  
C. 11110100101      D. 11111101101

5. 在计算机, 字符编码通常采用( C )

- A. 原码      B. 反码      C. ASCII 码      D. 补码

6. 设栈 S 的初始状态为空, 元素 a、b、c、d、e 依次入栈, 以下出栈序列不可能出现的有( C )。

- A. a、b、c、e、d      B. b、c、a、e、d  
C. a、e、c、b、d      D. d、c、e、b、a

7. 设栈 S 和队列 Q 的初始状态为空, 元素 e1, e2, e3, e4, e5, e6 依次通过栈 S, 一个元素出栈后即进入队列 Q。若出队的顺序为 e2, e4, e3, e6, e5, e1, 则栈 S 的容量至少应该为( B )

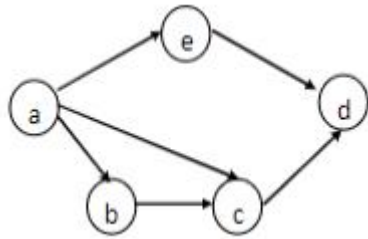
- A. 2      B. 3      C. 4      D. 5

8. 十进制数 -103 的补码是(A)。

- A. 10011001      B. 11100111      C. 10110011      D. 00011001

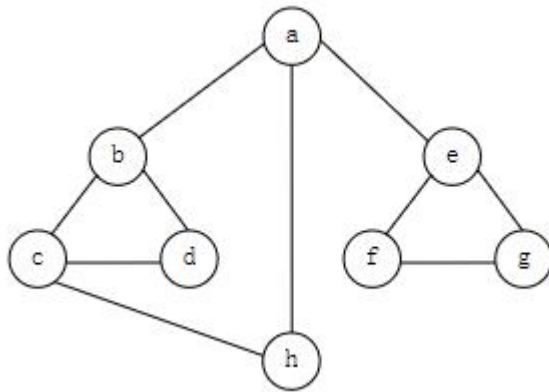
9. 对右图进行拓扑排序，可以得到不同的拓扑序列的个数是 (B)。

- A. 4      B. 3      C. 2      D. 1

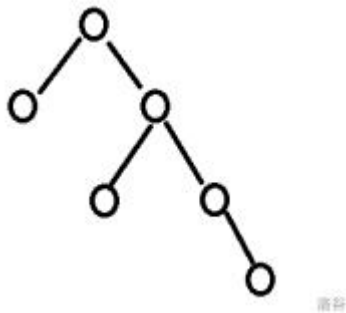


10. 若对如下无向图进行遍历，则下列选项中，不是广度优先遍历序列的是 (D)

- A. h, c, a, b, d, e, g, f  
 B. e, a, f, g, b, h, c, d  
 C. d, b, c, a, h, e, f, g  
 D. a, b, c, d, h, e, f, g



11. 一棵二叉树如右图所示，若采用顺序存储结构，即用一维数组元素存储该二叉树中的结点（根结点的下标为 1，若某结点的下标为  $i$ ，则其左孩子位于下标  $2i$  处、右孩子位于下标  $2i+1$  处），则该数组的最大下标至少为 ( C )。



- A. 6  
 B. 10  
 C. 15  
 D. 12

12. 像中国的回文联“洞帘水挂水帘洞，山果花开花果山”一样，如果将一个数的数字倒排后所得的数仍是这个数，这样的数称为回文数，例如 11，22，343，565，1881，20102 等，在所有三位数中回文数共有（ B ）。

A. 81 个    B. 90 个    C. 99 个    D. 100 个

13. 某高校开设 A 类选修课四门，B 类选修课三门。小刘从中共选取四门课程，若要求两类课程各至少选一门，则选法有（ D ）。

A. 18 种    B. 22 种    C. 26 种    D. 34 种

14. 某个车站虽狭长形，宽度只能容下一台车，并且只有一个出入口。已知某时刻该车站状态为空，从这一时刻开始的出人记录为：“进，出，进，进，进，出，出，进，进，进，出”。假设车辆入站的顺序为 1, 2, 3, …，则车辆出站的顺序为（ C ）。

A. 1、2、3、4、5

B. 1, 2, 4, 5, 7

C. 1, 4, 3, 7, 6

D. 1, 4, 3, 7, 2

15. 由 3 个 a, 5 个 b 和 2 个 c 构成的所有字符串中，包含子串“abc”的共有（ D ）个。

A. 40320

B. 39600

C. 840

D. 780

二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题 2 分，选择题 3 分，合计 30 分）

(1)

```
01 #include <iostream>
02 using namespace std;
03 void hanoi(int n, char start, char pass1, char pass2, char end) {
04     if (n > 2) hanoi(n - 2, start, end, pass2, pass1);
05     if (n > 1) printf("%c -> %c\n", start, pass2);
06     printf("%c -> %c\n", start, end);
07     if (n > 1) printf("%c -> %c\n", pass2, end);
08     if (n > 2) hanoi(n - 2, pass1, start, pass2, end);
09 }
10 int main() {
11     int n;
12     scanf("%d", &n);
13     hanoi(n, 'A', 'B', 'C', 'D');
14     return 0;
15 }
```

判断题

1. 把第 4 行到第 8 行的代码所有 "pass1" 替换为 "pass2", 所有 "pass2" 替换为 "pass1", 对于程序的正确性不会有影响 ( )
2. 当输入的  $n=3$  时, 会有 8 行的输出 ( )
3. 把第 3 行函数声明的所有 char 类型的参数改为 int 类型, 对于程序的正确性不会有影响 ( )

#### 单选题

4. 把第 5 行和第 7 行的 "if ( $n > 1$ )" 去掉, 在输入的正数  $n$  为 ( ) 时 对结果没有影响  
A. 偶数      B. 平方数      C. 质数      D. 立方数
5. 当输入的  $n=10$  时, 会有 ( ) 行的输出  
A. 93      B. 94      C. 95      D. 96
6. 程序的时间复杂度是 ( )  
A.  $O(n)$       B.  $O(n \log n)$       C.  $O(n^2)$       D.  $O(2^n)$

该程序是输出 4 柱汉诺塔问题的解法, 第 4 行表示把前  $n-2$  个柱子放到转移柱 1, 第 5 行把第  $n-1$  个盘子放到转移柱 2 上, 第 6 行把第  $n$  个盘子放到目标柱上, 第 7 行表示把第  $n-1$  一个盘子从转移柱 2 移到目标柱上, 第 8 表示把前  $n-1$  个盘子放到目标柱上。

1. ✓ 相当于交换了两个中间转移柱, 对输出的方案正确性没有影响。
2. × 5 行
3. ✓ 调用处是字符和输出处是 "%c", 就能正常输出
4. A 每次递归调用  $n$  会减 2, 所以偶数不会受影响
5. A ,  $a[2]=3$ , 当  $n$  是偶数时,  $a[n+2]=2a[n]+3$ ;
6. D 和三柱汉诺塔复杂度只有常数级的差别

(2)

```

01 #include <iostream>
02 using namespace std;
03 const int maxn = 100;
04 int k;
05 int a[maxn] = { 0 }, b[maxn] = { 0 };
06
07 void add(int* a, int* b) {
08     int jw = 0;
09     for (int i = 0; i < maxn; i++) {
10         b[i] = b[i] + a[i] + jw;
11         jw = b[i] / k;
12         b[i] = b[i] % k;
13     }
14 }
15
16 void multi(int* a, int c) {
17     int jw = 0;
18     for (int i = 0; i < maxn; i++) {

```

```

19     a[i] = a[i] * c + jw;
20     jw = a[i] / k;
21     a[i] = a[i] % k;
22 }
23 }
24
25 int main() {
26     int n;
27     cin >> n >> k;
28     a[1] = 1;
29     for (int i = 1; i <= n; i++) {
30         multi(a, i);
31         add(a, b);
32     }
33     int flag = 0;
34     for (int i = maxn-1; i > 0 ; i--) {
35         if (b[i] != 0) {
36             flag = i;
37             break;
38         }
39     }
40     char c;
40     for (int i = flag; i >= 1; i--) {
41         if (b[i]<10) c=' 0' +b[i];
42         else c=' A' +b[i];
43         cout << c;
44     }
45     if (b[0]<10) c=' 0' +b[0];
46     else c=' A' +b[i];
47     cout << c;
48     return 0;
49 }

```

#### 判断题

1. 输出的结果的第一个字符一定不为'0' ( )
2. 当输入的 k=5 时，输出只有数字没有字母 ( )
3. 当输入的 n=100, k=10 时，程序运行时会数组下标越界 ( )

#### 单选题

4. 当输入 n=5, k=10 时，输出结果是( )  
A. 1      B. 15      C. 120    D. 153
5. 当输入 n=10, k=8 时，输出结果是 ( )  
A. 4037913      B. 17316431      C. 3628800      D. 15657400
6. 当程序的两次输入 k 都是 17, n 的输入分别是 ( ), 输出的最后一个字符相同  
A. n=3 和 n=9      B. n=2 和 n=16    C. n=17 和 n=18    D. n=1 和 n=2

求  $1!$  到  $n!$  的和 (高精度), 输出的数字在  $k$  进制下表示, 输出只保留后 100 位, 并且输出时去掉前缀 0。

1. ☐ 当  $n=0$ , 输出的是 0
2. ☒ 5 进制时, 输出只会是 0-4
3. ☐ 虽然实际求  $1!$  到  $100!$  的和答案结果一定超过 100 位 ( $\text{maxn}$ ), 但是程序并不会下标越界
4. ☐ D
5. ☐ B 注意输出是 8 进制

6.C 17 是质数, 只有当  $17!$  开始, 每次新加的数字在 17 进制下末位为 0。(需要两个数都大于 16)

### 三、完善程序 (单选题, 每小题 3 分、合计 30 分)

(优美的进制) 问题: 给出整数  $n$ ;  $k$  进制是优美的, 当且仅当  $n$  在  $k$  进制下至少有两位, 且每一位的数值都不同。求对于给定的  $n$ , 有哪些进制是优美的, 不存在则输出 -1。试补全程序。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int MAXN = 100000;
4  int n;
5  int vis[MAXN], a[MAXN];
6  vector<int> ans;
7  int check(int k) {
8      int x = n, top = 0;
9      for (int i = 0; i <= k; i++) vis[i] = 0;
10     while (①) {
11         a[++top] = ②;
12         x = ③;
13     }
14     if (top < 2)
15         return 0;
16     for (int i = 1; i <= top; i++) {
17         if (④)
18             return 0;
19         vis[a[i]] = 1;
20     }
```

```

21         return 1;
22     }
23     int main() {
24         cin >> n;
25         for (int i = ⑤; i <= n; i++) {
26             if (check(i))
27                 ans.push_back(i);
28         }
29         if (ans.empty()) {
30             cout << -1;
31         }
32         for (int i = 0; i < ans.size(); i++)
33             cout << ans[i] << " ";
34         return 0;
35     }

```

34. ①处应填( A )

A.  $x > 0$                   B.  $x > 1$                   C.  $x / k > 0$                   D.  $x / k > 1$

35. ②处应填( B )

A.  $x / k$                                   B.  $x \% k$   
 C.  $(x - 1) / k + 1$                       D.  $(x - 1) \% k + 1$

36. ③处应填( A )

A.  $x / k$                                   B.  $x \% k$   
 C.  $(x - 1) / k + 1$                       D.  $(x - 1) \% k + 1$

37. ④处应填( D )。

A. `vis[i] == 1`                              B. `vis[a[i]] == 0`  
 C. `vis[i] == 0`                              D. `vis[a[i]] == 1`

38. ⑤处应填( C )。

A. 1                                  B.  $n - 1$                                   C. 2                                  D. 0

(扩充最小生成树) 给定一棵树，需要把树扩充成完全图，要求扩充后的图进行求解最小生成树的结果必然是原来的树。求扩充后的完全图的所有的边的权值总和。试补全 Kruskal 算法。

```

01 #include<iostream>
02 #include<algorithm>
03 using namespace std;
04 const int maxn=10000;

```

```

05 int n;
06 struct edge {
07     int a, b;
08     int len;
09     bool operator < (edge e2) {
10         return len < e2.len;
11     }
12 }list[maxn];
13 int group[maxn];
14 int cnt[maxn] ;
15 int find(int x) {
16     if (x != group[x]) {
17         group[x] = ①;
18     }
19     return group[x];
20 }
21 int main() {
22     cin>>n;
23     long long ans = 0;
24     for (int i = 0; i <= n - 1; i++) {
25         cin >> list[i].a >> list[i].b >> list[i].len;
26         ans+=list[i].len;;
27     }
28     ②;
29     for (int i = 1; i <= n; i++) {
30         group[i] = i;
31         cnt[i] = ③;
32     }
33     for (int i = 0; i < n - 1; i++) {
34         ans += (list[i].len + 1) * ④;
35         cnt[find(list[i].b)] += cnt[find(list[i].a)];
36         ⑤;
37     }
38     cout << ans << endl;
39     return 0;
40 }

```

1. ①处应填( )

- |                   |                     |
|-------------------|---------------------|
| A. find(group[x]) | B. group[x]+1       |
| C. find(x+1)      | D. find(group[x+1]) |

2. ②处应填( )

- |                                 |                             |
|---------------------------------|-----------------------------|
| A. sort(list + 1, list + n)     | B. sort(list, list + n)     |
| C. sort(list + 1, list + n + 1) | D. sort(list, list + n - 1) |



3. ③处应填( )

- A. 0     B. 1     C. i     D. n

4. ④处应填( )

- A. `cnt[list[i].a] * cnt[list[i].b]`  
B. `cnt[find(list[i].a)] * cnt[find(list[i].b)]`  
C. `cnt[list[i].a] * cnt[list[i].b] - 1`  
D. `cnt[find(list[i].a)] * cnt[find(list[i].b)] - 1`

5. ⑤处应填( )

- A. `group[list[i].a] = group[group[list[i].b]]`  
B. `group[find(list[i].b)] = group[find(list[i].a)]`  
C. `group[find(list[i].a)] = group[find(list[i].b)]`  
D. `group[group[list[i].b]] = group[list[i].a]`

【2612】通过 Kruskal 算法模拟生成树的构建进行分治，在并查集进行合并的时候，除了生成树内的那条边以外，两个需要合并的集合之间的点对的边的权值都需要比那条边权值大。

1. A 并查集基础知识
2. D 把  $n-1$  条边排序，注意 24 行下标是从 0 开始
3. B 并查集基础知识，`cnt` 数组记录的是集合的元素数量，初始每个点单独一个集合，集合数量为 1
4. D 因为需要并查集函数取查到真正的集合编号（套上 `find` 函数）。除了生成树内的那条边以外（需要-1）
5. C 35 行可知是 `list[i].b` 的集合数量加上 `list[i].a` 的集合数量，所以是 `a` 集合往 `b` 集合处合并。